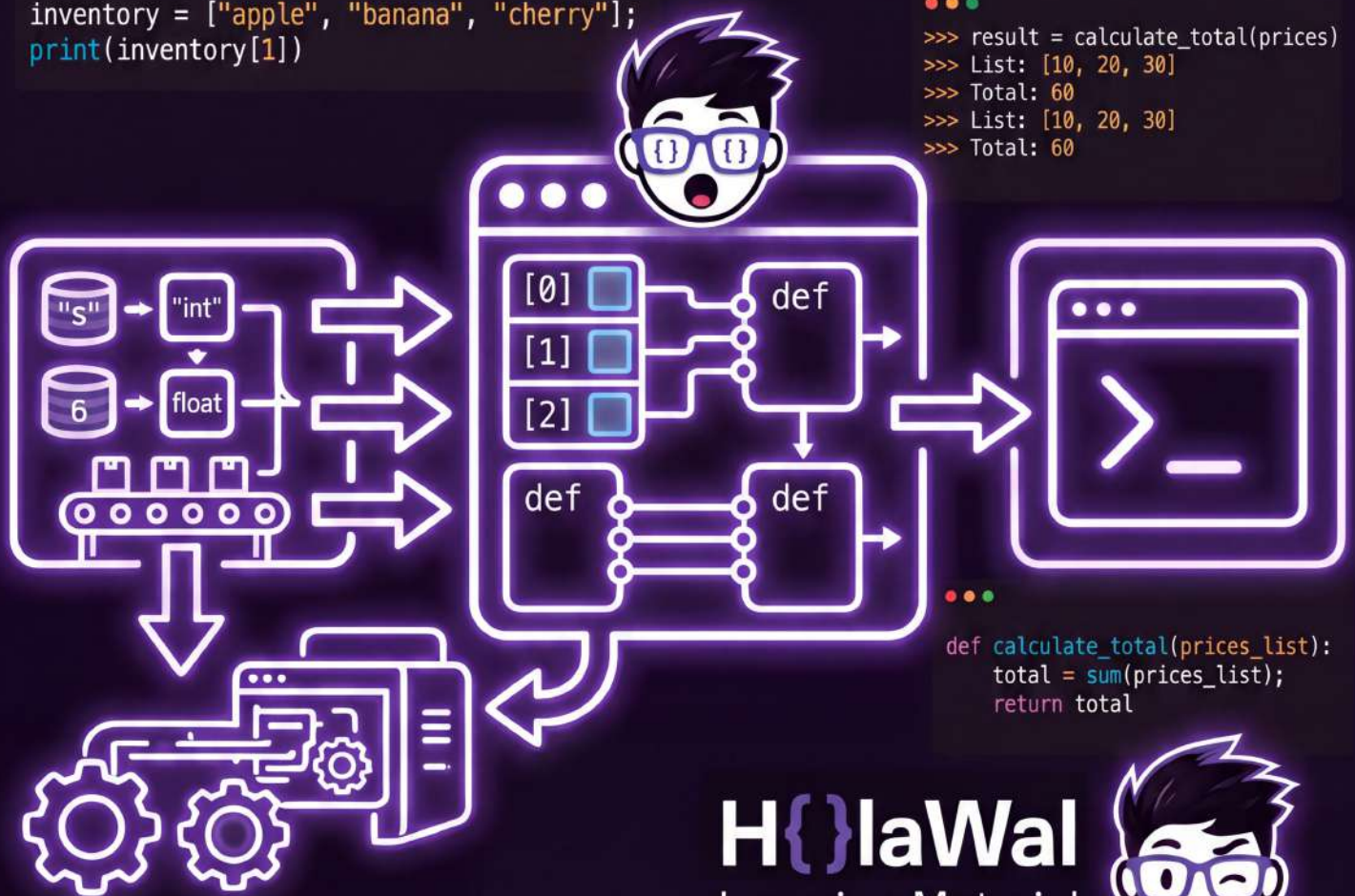


MÓDULO 4: INTRODUCCIÓN A PYTHON

Datos y Modularidad — Listas y
Funciones Reutilizables (def)

```
inventory = ["apple", "banana", "cherry"];  
print(inventory[1])
```

```
>>> result = calculate_total(prices)  
>>> List: [10, 20, 30]  
>>> Total: 60  
>>> List: [10, 20, 30]  
>>> Total: 60
```



```
def calculate_total(prices_list):  
    total = sum(prices_list);  
    return total
```



Módulo 4: Colecciones y Modularidad

Listas, Métodos de Estructuras de Datos y Funciones Reutilizables (def)

Propósito Pedagógico: Transicionar desde scripts lineales y variables aisladas hacia programas organizados, modulares y escalables mediante la combinación de estructuras de datos y funciones.

[OBJETIVOS] Objetivos Clave de la Sesión

- **Comprender colecciones indexadas:** Almacenar y manipular grupos de datos dinámicos mediante listas [].
- **Dominar los métodos esenciales:** Agregar, insertar, remover y extraer elementos con eficiencia.
- **Automatizar recorridos:** Iterar sobre colecciones con bucles for aplicando transformaciones de datos.
- **Modularizar código con def:** Encapsular lógica, recibir parámetros y devolver valores con return.
- **Integrar arquitectura limpia:** Construir un Gestor de Tareas interactivo desacoplado en funciones.

1. ¿Qué es una Lista y Cómo Funciona el Tren de Datos?

Hasta ahora, una variable simple funcionaba como una caja que solo guardaba un único valor (ej. edad = 25). Si necesitas administrar 500 productos o 50 alumnos, no crearás 50 variables separadas. Una **Lista** es una colección ordenada y modificable delimitada por corchetes [] y separada por comas.

Vagón [0]	Vagón [1]	Vagón [2]	Vagón [-1] (Último)
"Ana"	"Carlos" -> "Christian"	"Beatriz"	"David"
Índice 0 (Primer elemento)	Índice 1 (Modificable)	Índice 2 (Tercer elemento)	Índice -1 / Posición final

```
# Creación, lectura y modificación de una lista
alumnos = ["Ana", "Carlos", "Beatriz", "David"]

print(alumnos[0]) # Imprime 'Ana' (Primer elemento)
print(alumnos[2]) # Imprime 'Beatriz' (Tercer elemento)
print(alumnos[-1]) # Imprime 'David' (Último elemento)

# Modificación de un elemento existente:
alumnos[1] = "Christian" # Reemplaza 'Carlos' por 'Christian'
print(alumnos)           # ['Ana', 'Christian', 'Beatriz', 'David']
```

[ALERTA] Alerta Técnica: IndexError (Índice Fuera de Rango)

Si una lista tiene 4 elementos, los índices válidos van del **0 al 3**. Si intentas consultar alumnos[4], Python arrojará IndexError: list index out of range. ¡El índice del último elemento es siempre **len(lista) - 1**!

2. Los 5 Superpoderes de las Listas (Métodos Esenciales)

Los métodos son acciones integradas en Python que permiten manipular y transformar colecciones de manera dinámica:

Método / Función	Descripción Pedagógica	Ejemplo de Código	Resultado en la Lista
<code>len(lista)</code>	Devuelve el conteo total de elementos.	<code>len(carrito)</code>	2 (entero)
<code>.append(elem)</code>	Agrega un nuevo elemento al final absoluto.	<code>carrito.append("Teclado")</code>	Queda al final
<code>.insert(pos, elem)</code>	Inserta en un índice exacto desplazando el resto.	<code>carrito.insert(1, "Audífonos")</code>	Ocupa la posición 1
<code>.remove(valor)</code>	Busca y elimina la primera coincidencia del valor.	<code>carrito.remove("Mouse")</code>	Elimina por coincidencia
<code>.pop(pos)</code>	Elimina y devuelve el elemento de la posición (o el último).	<code>item = carrito.pop()</code>	Extrae y retorna el elemento

```
# Demostración interactiva de métodos
carrito = ["Laptop", "Mouse"]
carrito.append("Teclado")      # ['Laptop', 'Mouse', 'Teclado']
carrito.insert(1, "Audífonos") # ['Laptop', 'Audífonos', 'Mouse', 'Teclado']
carrito.remove("Mouse")        # Elimina 'Mouse'
eliminado = carrito.pop()      # Quita 'Teclado' y lo guarda en la variable

print(f"Total de productos: {len(carrito)}") # Total: 2
print(f"Carrito actual: {carrito}")          # ['Laptop', 'Audífonos']
```

3. Recorrer Listas con el Bucle 'for': La Forma Pitónica

Python permite iterar sobre cada elemento directamente con la estructura `for item in lista:`, lo que simplifica cálculos acumulativos, filtros y transformaciones automáticas:

```
# Aplicación de descuentos y cálculo de total acumulado
precios = [120.0, 45.5, 80.0, 210.0]
total_con_descuento = 0.0

print("=== LISTA DE PRECIOS CON 10% DE DESCUENTO ===")
for precio in precios:
    precio_final = precio * 0.90 # Aplica 10% de descuento
    total_con_descuento += precio_final
    print(f"Precio original: ${precio:.2f} -> Con Desc: ${precio_final:.2f}")

print(f"\nTotal acumulado a pagar: ${total_con_descuento:.2f}")
```

4. Funciones con 'def' (Tus Propias Mini-Fábricas de Código)

Una **Función** es un bloque de código reutilizable e independiente que ejecuta una tarea específica. Permite no duplicar código y organizar la solución en módulos que reciben entradas y devuelven salidas.

Concepto en Python	Analogía: La Cafetera Automática	Rol en el Programa
<code>def mi_funcion():</code>	Instalar y configurar la cafetera en la cocina.	Crea y nombra la rutina de trabajo.
Parámetros / Argumentos	El agua y los granos de café que introduces.	Datos de entrada que requiere la función.
Cuerpo de la Función	Calentar el agua y filtrar los granos internamente.	Procesamiento y lógica interna.
<code>return resultado</code>	La taza de café lista entregada en la bandeja.	Devuelve el valor producido hacia afuera.
Invocación / Llamada	Presionar el botón de inicio de la cafetera.	Ejecuta la función pasándole valores reales.

```
# 1. Definición de la función con parámetros y return
def calcular_igv(precio_netto):
    impuesto = precio_netto * 0.18
    total = precio_netto + impuesto
    return total # Devuelve el valor calculado

# 2. Invocación de la función con distintos argumentos
precio_laptop = calcular_igv(1000.0) # total: 1180.0
precio_mouse = calcular_igv(50.0)    # total: 59.0

print(f"Total Laptop: ${precio_laptop:.2f} | Total Mouse: ${precio_mouse:.2f}")
```

5. Sinergia Profesional: Funciones que Procesan Listas

El estándar de desarrollo profesional consiste en diseñar funciones modulares capaces de recibir listas completas de datos, procesarlas internamente mediante bucles y retornar métricas consolidadas o reportes:

```
# Función que procesa una lista de calificaciones
def analizar_calificaciones(lista_notas):
    if len(lista_notas) == 0:
        return 0.0, 0 # Retorna promedio 0 y 0 aprobados

    suma = 0.0
    aprobados = 0
    for n in lista_notas:
        suma += n
        if n >= 11:
            aprobados += 1

    promedio = suma / len(lista_notas)
    return promedio, aprobados

# Ejecución con datos reales:
notas_salon_a = [16, 8, 14, 11, 19, 9]
prom, aprob = analizar_calificaciones(notas_salon_a)
print(f"Promedio: {prom:.2f} | Aprobados: {aprob}/{len(notas_salon_a)}")
```

6. Proyecto Guiado Integrador: 'Gestor de Tareas Modular'

Evolución de Arquitectura: Desacoplamos las responsabilidades en 3 funciones independientes (mostrar_tareas, agregar_tarea y eliminar_tarea) orquestadas por un bucle principal while True:

```
# =====
# PROYECTO SESIÓN 4: GESTOR DE TAREAS MODULAR
# =====
def mostrar_tareas(lista):
    if len(lista) == 0:
        print("\n[!] No tienes tareas pendientes.")
    else:
        print(f"\nTienes {len(lista)} tarea(s) pendiente(s):")
        for i, t in enumerate(lista, start=1):
            print(f"  {i}. [ ] {t}")

def agregar_tarea(lista, nueva_tarea):
    lista.append(nueva_tarea)
    print(f"-> Tarea '{nueva_tarea}' registrada exitosamente.")

def eliminar_tarea(lista, numero):
    if 1 <= numero <= len(lista):
        borrada = lista.pop(numero - 1)
        print(f"-> Tarea '{borrada}' completada y eliminada.")
    else:
        print("[!] Número de tarea inválido.")

# --- Bucle Principal del Sistema ---
mis_tareas = ["Estudiar Python", "Hacer ejercicio"]
while True:
    print("\n=== GESTOR DE TAREAS MODULAR ===")
    print("1. Ver tareas | 2. Agregar | 3. Completar | 4. Salir")
    opc = input("Seleccione opción (1-4): ")
    if opc == "1":
        mostrar_tareas(mis_tareas)
    elif opc == "2":
        t = input("Nombre de la nueva tarea: ")
        agregar_tarea(mis_tareas, t)
    elif opc == "3":
        mostrar_tareas(mis_tareas)
        if len(mis_tareas) > 0:
            num = int(input("Número de tarea a completar: "))
            eliminar_tarea(mis_tareas, num)
    elif opc == "4":
        print("\nGracias por organizar tu día. ¡Hasta la próxima!")
        break
    else:
        print("[!] Opción no válida.")
```

7. Banco de Ejercicios y Retos Prácticos Graduales

Nivel y Reto	Consigna y Requisitos Técnicos	Estructura Sugerida
Nivel 1 (Fácil) Invitados VIP	Crea una lista con 4 amigos. Pide un nombre con input(), agrégalo con .append() e imprime la lista y el total con len().	<code>lista.append(...)</code> <code>len(lista)</code>
Nivel 2 (Fácil) Función Saludo	Crea una función saludar(nombre) que reciba un nombre como parámetro e imprima un mensaje cálido de bienvenida.	<code>def saludar(n):</code> <code>print(...)</code>
Nivel 3 (Intermedio) Calculadora Descuento	Crea una función aplicar_descuento(precio, porcentaje) que calcule y devuelva mediante return el precio final rebajado.	<code>def aplicar_desc(p, pct):</code> <code>return p*(1-pct/100)</code>
Nivel 4 (Avanzado) Buscador Modular	Crea una función buscar_contacto(lista, nombre). Si está en la lista (usando if nombre in lista:), indica su posición; si no, ofrece agregarlo.	<code>def buscar_contacto(l, n):</code> <code>if n in l: ... else: ...</code>

[RUMBO AL DÍA 5] ¡Todo Listo para el Gran Final (Día 5: Asistente Virtual Inteligente)!

Has consolidado con éxito los **5 pilares fundamentales** de Python: **1. Variables** | **2. Decisiones if** | **3. Bucles while/for** | **4. Listas []** | **5. Funciones def**.

En la **Sesión 5** integraremos todo para crear un **Asistente Virtual por Consola**.